



32. Python: Matplotlib

Turning numbers into stories, one graph at a time.

Previous Sections:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 31. Python: Pandas Series](#)

[!\[\]\(d66ff64371a51729ac8c1cdaa685ba6f_img.jpg\) 31.1. Python: Pandas DataFrame](#)

When people think about Python, they often think about automation, Artificial Intelligence, or web development. But one of Python's most important superpowers is something much simpler: **Helping us see data.**

Imagine opening a spreadsheet with thousands of rows of numbers. Technically, the information is there, but understanding it can be difficult. Human brains are naturally better at spotting patterns visually than reading endless tables. This is where **Data Visualization** comes in.

Data Visualization is the process of transforming raw data into visual representations such as: Charts; Graphs; Figures; Maps; Dashboards

Instead of staring at numbers, we can immediately see trends, relationships, outliers, and patterns. For example:

- Is sales revenue increasing?
- Which product performs best?
- Are two variables related?
- Is the data evenly distributed?

A well-designed graph can often communicate information faster than an entire page of statistics. This is why data visualization has become a fundamental skill in Data Science, Data Analytics, Machine Learning, Business Intelligence, and scientific research.

In the Data Analytics course, we will focus heavily on **interpreting and analyzing graphs**. Here, however, our goal is different. We will focus on the Python libraries that allow us to create those visualizations in the first place. And among all visualization libraries available in Python, two stand above the rest:

- **Matplotlib** — the foundation of Python plotting
- **Seaborn** — a higher-level library that makes beautiful statistical visualizations much easier

Together, these two libraries form the backbone of most Python-based data visualization workflows.

This section focuses on the Matplotlib library

1. Introduction to Matplotlib

Before Seaborn, before fancy dashboards, and before most modern visualization tools in Python, there was **Matplotlib**.

Matplotlib is one of the most powerful and widely used data visualization libraries in Python. It provides everything needed to create visual representations of data, ranging from simple line charts to complex 3D visualizations.

Think of Matplotlib as the **drawing engine** behind many other visualization libraries.

Its strengths include:

- Simple and beginner-friendly syntax
- Extremely flexible customization
- Support for both 2D and 3D plots
- Integration with NumPy, Pandas, and many scientific libraries
- A massive community and extensive documentation

One reason Matplotlib remains so popular is that it gives developers precise control over nearly every aspect of a graph: Titles; Labels; Colors; Axes; Legends; Markers; Layouts

If a graph can be customized, Matplotlib can probably do it.

In practice, many other visualization libraries—including Seaborn—are built on top of Matplotlib. Learning Matplotlib first helps you understand how Python plotting works underneath the hood.

To use Matplotlib, we typically import its plotting module:

```
import matplotlib.pyplot as plt
```

The `pyplot` module provides a collection of functions for creating and displaying graphs.

Throughout this chapter, most visualizations will begin with this import statement. Because before we can analyze data, we first need a way to draw it.

2. Common Operations in Matplotlib

Now that we know how to import Matplotlib, it's time to learn the tools you'll use almost every time you create a graph.

Think of these as your **Matplotlib survival kit**. You don't need to memorize everything immediately. Most developers learn these naturally after creating a few plots.

2.1. Matplotlib Cheatsheet

The following functions are among the most commonly used in Matplotlib:

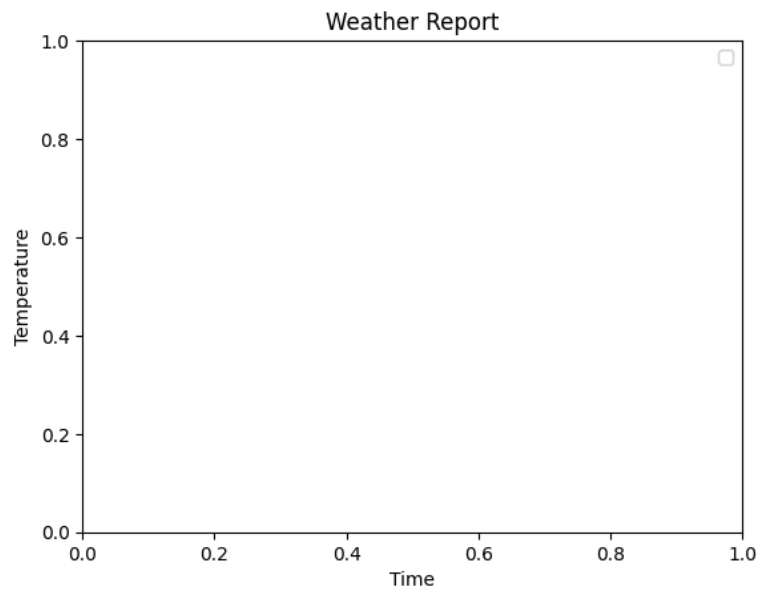
Function	Most Useful Parameters	Purpose / Action
<code>figure()</code>	<code>num, figsize, dpi, facecolor, edgecolor, frameon</code>	Creates a new window/canvas; sets dimensions, resolution, and background colors.
<code>show()</code>	<i>None</i>	Displays the completed graph on your screen.
<code>text()</code>	<code>x, y, s, fontsize, color, ha, va</code>	Adds text string <code>s</code> at coordinate position (<code>x</code> , <code>y</code>).
<code>xlabel()</code>	<code>xlabel, fontsize, fontweight, pad</code>	Adds a text label to the horizontal x-axis.

Function	Most Useful Parameters	Purpose / Action
<code>ylabel()</code>	<code>ylabel, fontsize, fontweight, pad</code>	Adds a text label to the vertical y-axis.
<code>title()</code>	<code>label, fontsize, loc, pad, weight</code>	Adds a main header title above the plot area.
<code>clear()</code>	<i>None</i> (or use <code>clf()</code> / <code>cla()</code>)	Clears the current active axis or figure window.
<code>savefig()</code>	<code>fname, dpi, bbox_inches, transparent</code>	Saves your figure directly to an image file (e.g., PNG).
<code>legend()</code>	<code>loc, bbox_to_anchor, title, fontsize</code>	Displays the color-coded guide legend for your data.
<code>tight_layout()</code>	<code>pad, h_pad, w_pad</code>	Automatically adjusts subplots to fit labels without clipping.
<code>grid()</code>	<code>visible, which, axis, color, linestyle, linewidth, alpha</code>	Toggles or styles the background measurement grid lines.

Example:

```
plt.xlabel("Time")
plt.ylabel("Temperature")
plt.title("Weather Report")
plt.legend()
plt.show()
```

This code only returns an empty plot



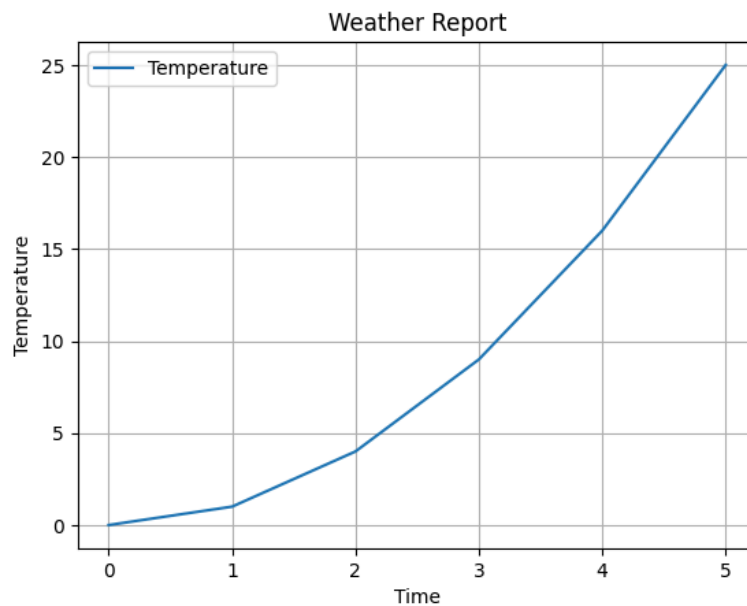
```
import matplotlib.pyplot as plt

time = [0, 1, 2, 3, 4, 5]
temperature = [0, 1, 4, 9, 16, 25]

plt.plot(time, temperature, label="Temperature")

plt.xlabel("Time")
plt.ylabel("Temperature")
plt.title("Weather Report")
plt.grid()

plt.legend()
plt.show()
```



You will see many of these functions appearing repeatedly throughout this section.

2.2. Line Graphs

When people think of graphs, this is usually the first thing that comes to mind. A **line graph** connects data points together using lines, making it useful for showing:

- Trends over time
- Growth and decline
- Continuous relationships
- Changes between observations

Despite the name, a line graph doesn't always look perfectly straight. Depending on your data, it may appear as a straight line, a zig-zag line, a smooth-looking curve, a rapidly changing pattern. The graph simply connects points in the order they are given.

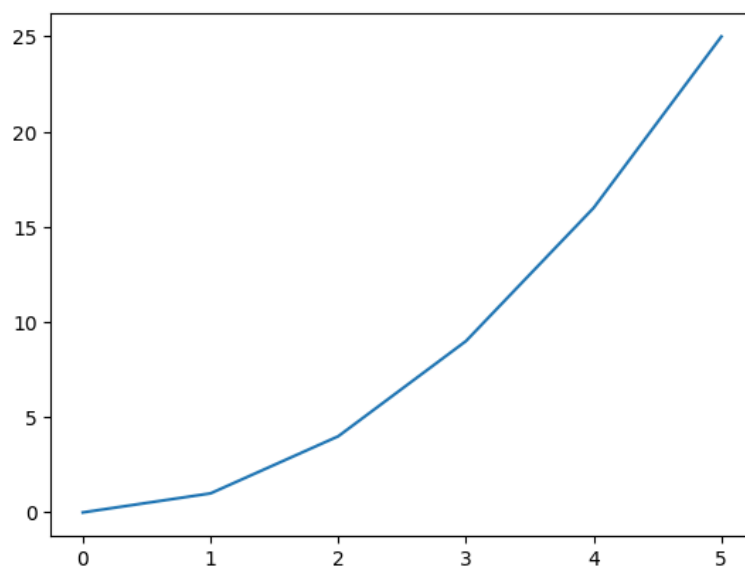
Let's start with some simple examples.

```
import matplotlib.pyplot as plt
```

```
# Create data
x_values = [0, 1, 2, 3, 4, 5]
y_values = [0, 1, 4, 9, 16, 25]

# Plot the graph
plt.plot(x_values, y_values)

# Display it
plt.show()
```



Here, Matplotlib draws a line connecting each `(x, y)` coordinate pair.

Notice that the values grow faster and faster because the y-values are squares: $(0^2, 1^2, 2^2, 3^2, 4^2, 5^2)$. The result looks somewhat curved even though we are using a line graph.

- **Inferring the x-values:** Sometimes you only provide y-values. In that case, Matplotlib automatically assumes the x-values are: `[0, 1, 2, 3, ...]` .
Example:

```
import matplotlib.pyplot as plt

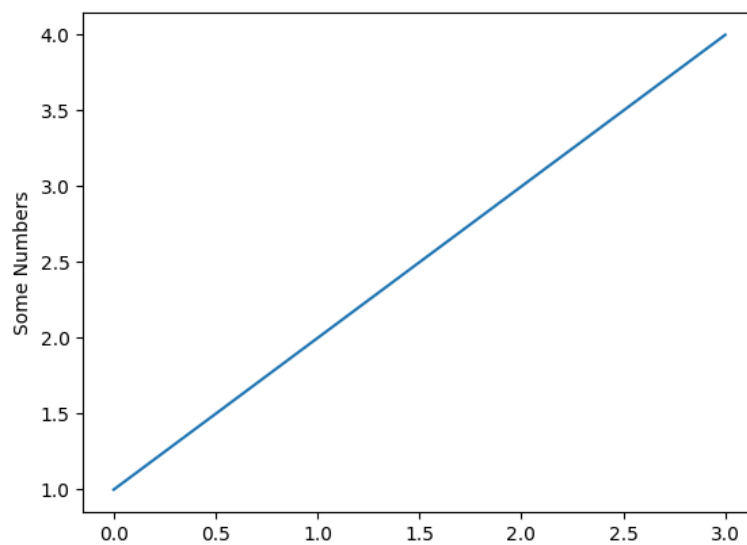
plt.plot([1, 2, 3, 4])
plt.ylabel("Some Numbers")
plt.show()
```

Internally, Matplotlib treats this as:

$x = [0, 1, 2, 3]$

$y = [1, 2, 3, 4]$

This shortcut is useful for quick visualizations.



- **Adding Labels and Titles:** A graph without labels is like a map without street names. People can see the picture. But they may not understand what they're looking at.


```
import matplotlib.pyplot as plt

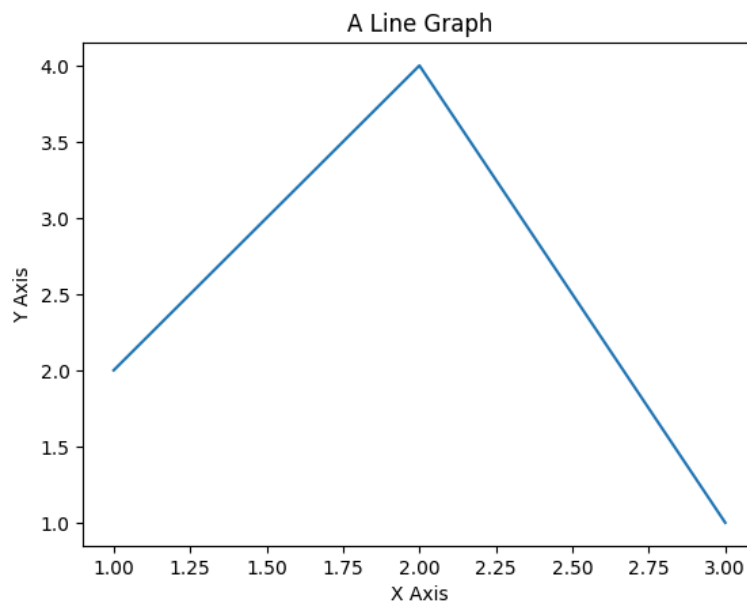
x = [1, 2, 3]
y = [2, 4, 1]

plt.plot(x, y)

plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.title("A Line Graph")
plt.show()
```

A few extra lines of code can make a graph significantly easier to understand.



- **Displaying Multiple Lines:** A graph can contain multiple datasets at once. This is useful when comparing products, sales figures, algorithms, and experimental results

```
import matplotlib.pyplot as plt

# First line
x1 = [1, 2, 3]
```

```

y1 = [2, 4, 1]

plt.plot(x1, y1, label="Line 1")

# Second line
x2 = [1, 2, 3]
y2 = [4, 1, 3]

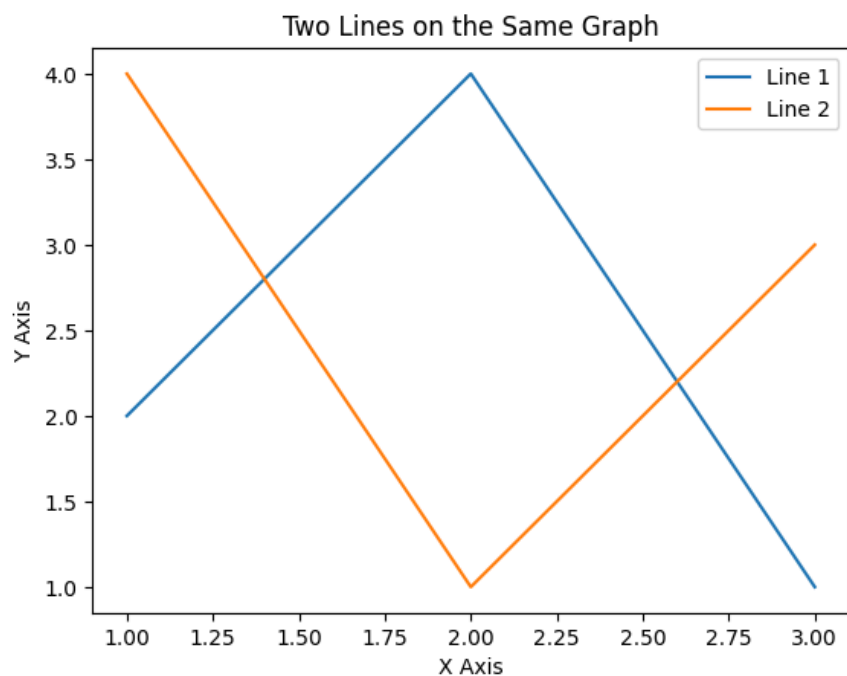
plt.plot(x2, y2, label="Line 2")

plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.title("Two Lines on the Same Graph")

plt.legend()
plt.show()

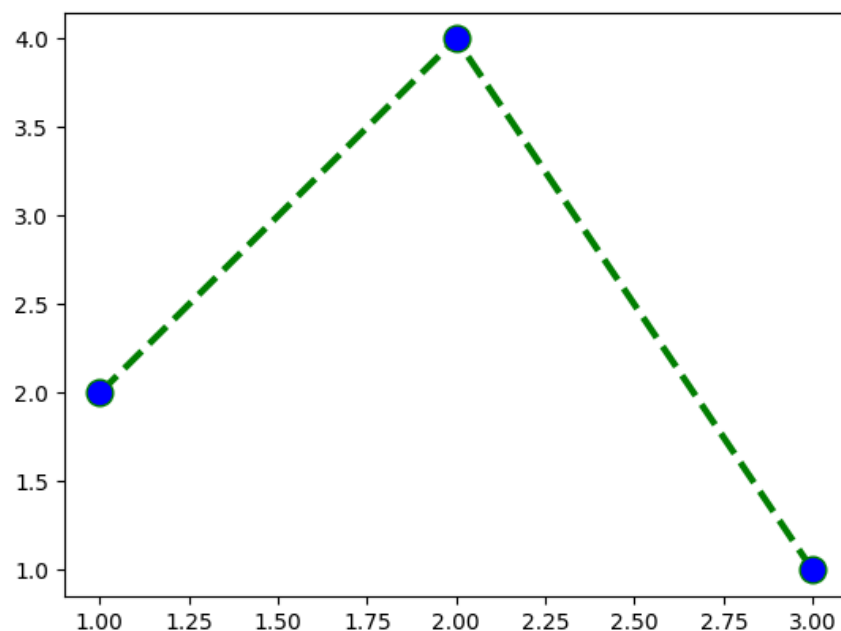
```



The `label` parameter gives each line a name. The `legend()` function then displays those names on the graph. Without a legend, viewers might not know which line represents which dataset.

- **Customizing a Line:** Matplotlib allows us to customize nearly every aspect of a graph.

```
plt.plot(x, y, color="green",  
         linestyle="dashed",  
         linewidth=3,  
         marker="o",  
         markerfacecolor="blue",  
         markersize=12)
```



This changes: Line color; Line style; Line thickness; Marker shape; Marker color; Marker size. A little customization can make graphs much easier to read and much more visually appealing.

- Marker Styles: Notice the `marker` parameter in the previous example. We are not limited to circles (`'o'`). Matplotlib provides many marker styles that can be used to highlight individual data points.

Some common markers include:

Marker	Shape
'o'	Circle
'^'	Triangle
's'	Square
'*'	Star
'D'	Diamond
'x'	Cross
'+'	Plus Sign

Example:

```
plt.plot(x, y, marker='o') # Circle
plt.plot(x, y, marker='^') # Triangle
plt.plot(x, y, marker='s') # Square
```

Matplotlib also supports a shorthand notation that combines color, line style, and marker into a single string.

```
plt.plot(x, y, 'ro') # Red circles
plt.plot(x, y, 'go') # Green circles
plt.plot(x, y, 'b^') # Blue triangles
plt.plot(x, y, 'rs') # Red squares
```

Think of these as quick styling shortcuts. They are convenient for simple plots, while the longer keyword approach (`color=` , `marker=` , `linestyle=`) provides greater control for complex visualizations.

This shorthand notation is one of the reasons Matplotlib is so popular—simple graphs can often be created with only a single line of code.

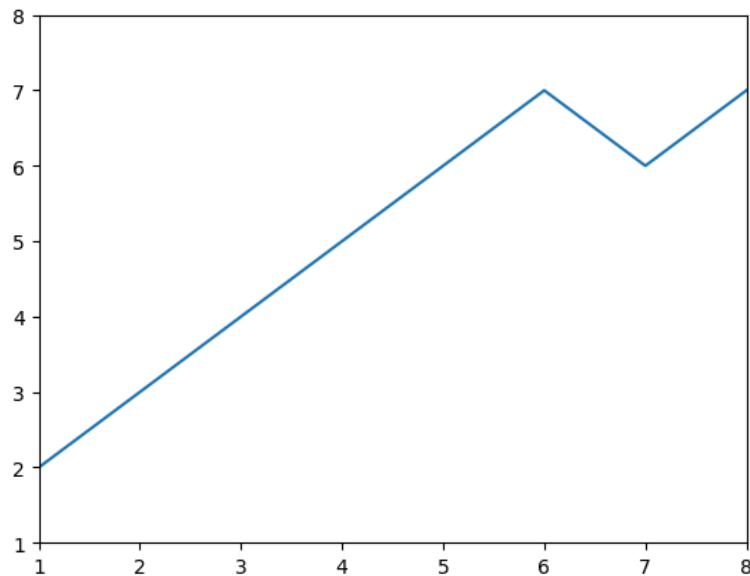
- Limiting the Axis Range: Sometimes a graph displays too much empty space. We can zoom in by restricting the axis ranges.

```
# Plot a simple line graph
x = [1, 2, 3, 5, 6, 7, 8]
y = [2, 3, 4, 6, 7, 6, 7]
```

```
plt.plot(x, y)

plt.xlim(1, 8)
plt.ylim(1, 8)

plt.show()
```



`plt.xlim(1, 8)` and `plt.ylim(1, 8)` tells Matplotlib to show only x-values between 1 and 8, and show only y-values between 1 and 8. Think of it as adjusting the camera lens on your graph.

2.3. Bar Graphs

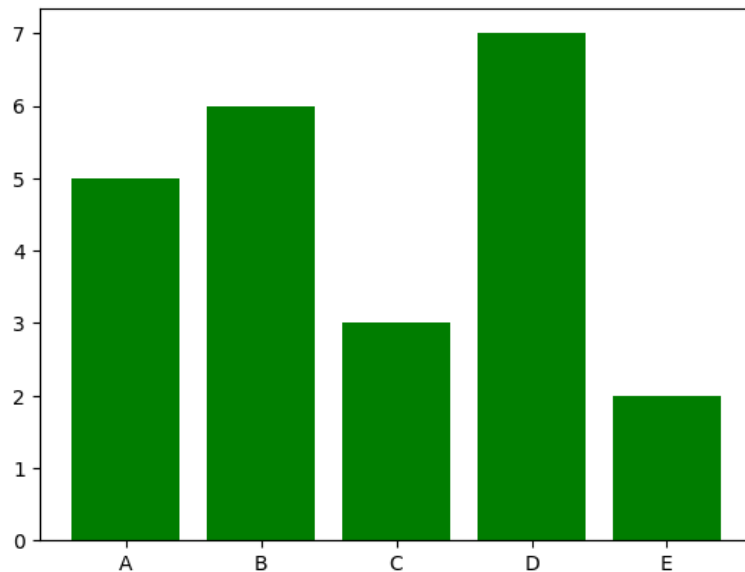
While line graphs are great for trends, bar graphs are excellent for comparing categories. Questions like: Which student scored highest? Which product sold the most? Which department spent the most money? are often easier to answer using a bar graph.

Example:

```
import matplotlib.pyplot as plt

values = [5, 6, 3, 7, 2]
names = ["A", "B", "C", "D", "E"]
```

```
plt.bar(names, values, color="green")
plt.show()
```



Each category is represented by a bar. The taller the bar, the larger the value. At a glance, we can immediately identify the highest and lowest categories.

You can also draw a horizontal bar graph:

```
import matplotlib.pyplot as plt

values = [5, 6, 3, 7, 2]
names = ["A", "B", "C", "D", "E"]

plt.barh(names, values, color="green")
plt.show()
```

Common Types of Bar Graphs

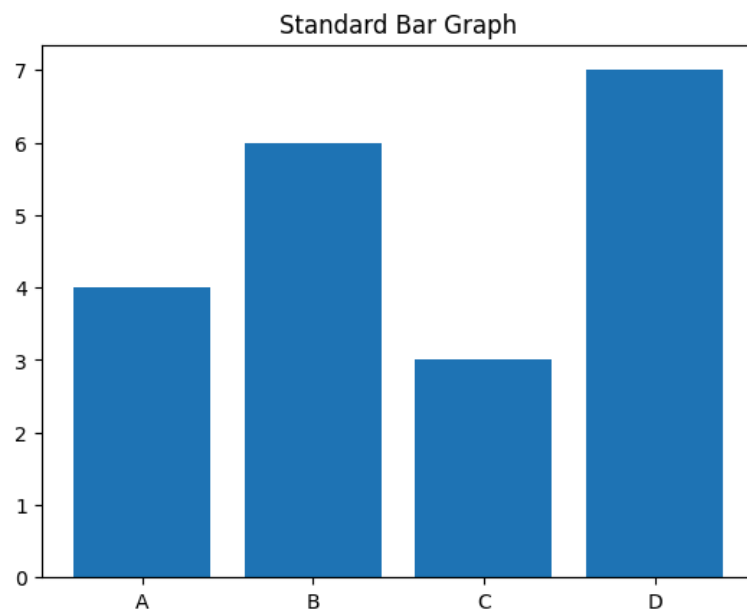
As datasets become larger, we often need more advanced variations of bar charts.

- **Standard Bar Graph:** The simplest form. Each category has one bar. It is used when comparing a single value across multiple categories.

```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C', 'D']
values = [4, 6, 3, 7]

plt.bar(categories, values)
plt.title("Standard Bar Graph")
plt.show()
```



- **Clustered Bar Graph:** Multiple bars are placed side-by-side. It is used for comparing multiple groups within the same category. Example: Male vs Female students; Product sales in different years; Revenue across multiple regions.

```
import numpy as np
import matplotlib.pyplot as plt

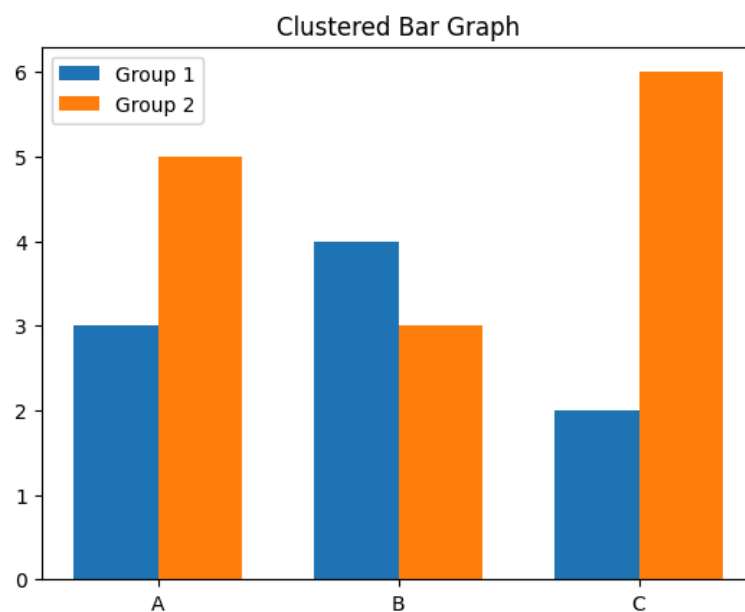
categories = ['A', 'B', 'C']
group1 = [3, 4, 2]
group2 = [5, 3, 6]

x = np.arange(len(categories))
```

```
width = 0.35

plt.bar(x - width/2, group1, width, label='Group 1')
plt.bar(x + width/2, group2, width, label='Group 2')

plt.xticks(x, categories)
plt.legend()
plt.title("Clustered Bar Graph")
plt.show()
```



- Stacked Bar Graph: Bars are stacked on top of each other. It is used to show total value with the contribution of each component. Examples are monthly expenses, population demographics, or market share composition

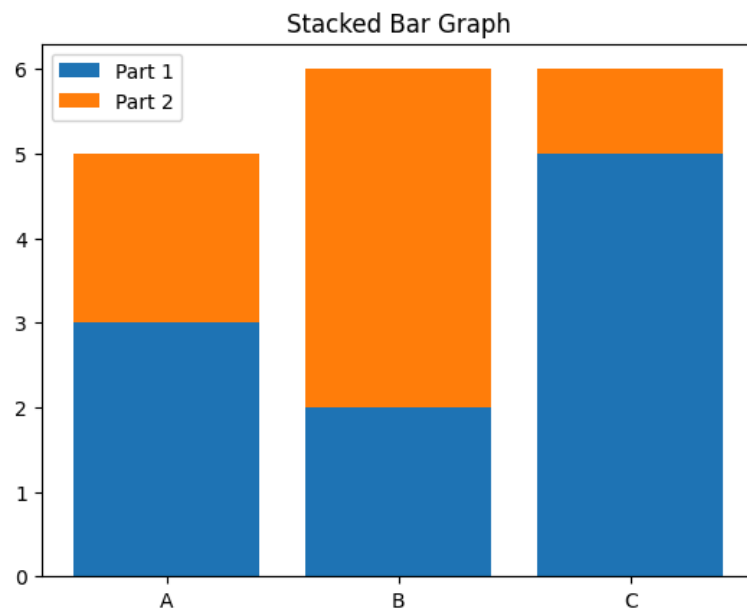
```
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C']
part1 = [3, 2, 5]
part2 = [2, 4, 1]

plt.bar(categories, part1, label='Part 1')
plt.bar(categories, part2, bottom=part1, label='Part 2')
```



```
plt.legend()
plt.title("Stacked Bar Graph")
plt.show()
```



- **Stacked Clustered Bar Graph:** The most advanced variation. It combines clustering and stacking, allowing comparisons between multiple groups while simultaneously showing composition within each group. These are common in professional dashboards, business reports, and analytics systems.

```
import numpy as np
import matplotlib.pyplot as plt

categories = ['A', 'B', 'C']

# Group 1 (stacked)
g1_part1 = [3, 2, 5]
g1_part2 = [2, 3, 1]

# Group 2 (stacked)
g2_part1 = [4, 1, 3]
```

```

g2_part2 = [1, 2, 2]

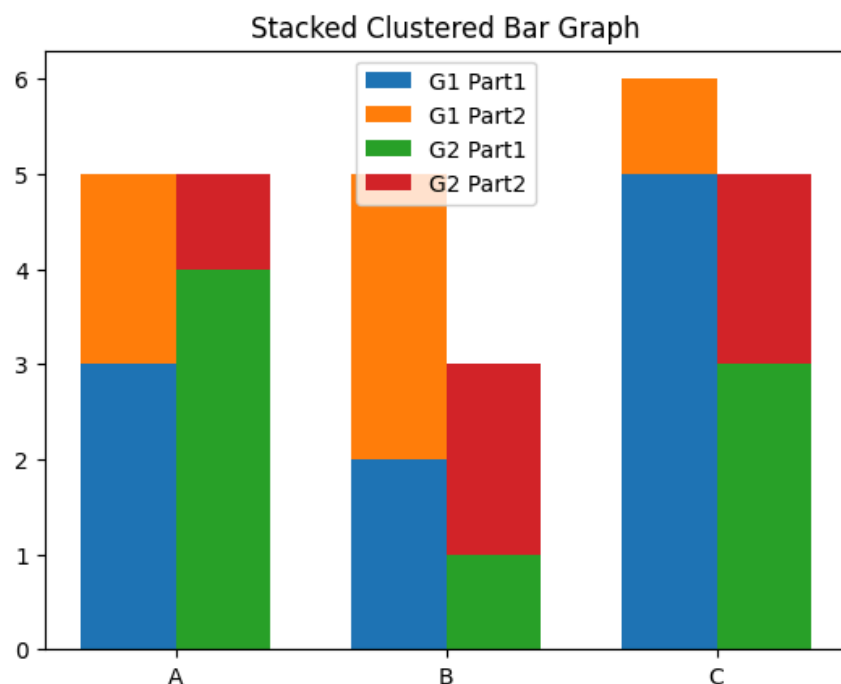
x = np.arange(len(categories))
width = 0.35

# Group 1 bars
plt.bar(x - width/2, g1_part1, width, label='G1 Part1')
plt.bar(x - width/2, g1_part2, width, bottom=g1_part1, label='G1 Part2')

# Group 2 bars
plt.bar(x + width/2, g2_part1, width, label='G2 Part1')
plt.bar(x + width/2, g2_part2, width, bottom=g2_part1, label='G2 Part2')

plt.xticks(x, categories)
plt.legend()
plt.title("Stacked Clustered Bar Graph")
plt.show()

```



2.4. Scatter Plots

Sometimes connecting points with lines doesn't make sense. What if the data points are independent observations? What if we simply want to see whether two variables are related? This is where **Scatter Plots** become useful.

A scatter plot displays individual points on a graph without connecting them together. Each point represents a pair of values `(x, y)`.

Scatter plots are commonly used for: Correlation analysis; Pattern detection; Outlier detection; Regression analysis; Machine Learning visualization

For example: Hours studied vs Exam score; House size vs House price; Height vs Weight. The goal is often to discover whether a relationship exists between two variables.

- Basic Scatter Plot

```
import matplotlib.pyplot as plt

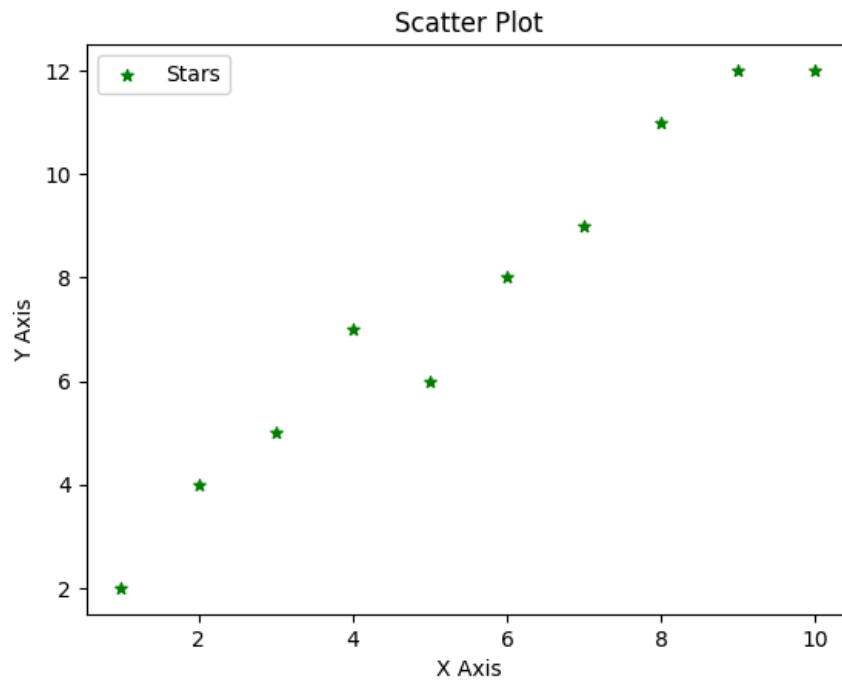
x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [2, 4, 5, 7, 6, 8, 9, 11, 12, 12]

plt.scatter(x, y, label="Stars", color="green", marker="*",
            s=30)

plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.title("Scatter Plot")
plt.legend()
plt.show()
```

Unlike `plot()`, which draws lines between points, `scatter()` displays every point individually.

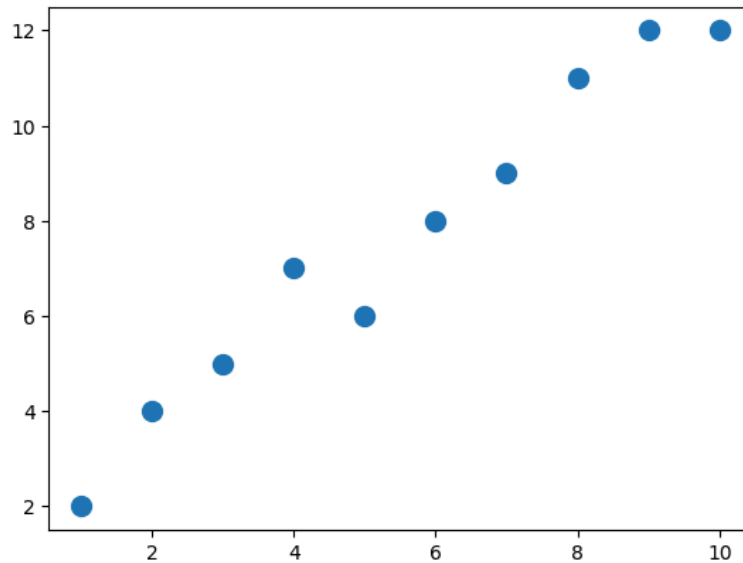


- Common Scatter Plot Markers: Just like line graphs, scatter plots support many marker styles:

Marker	Shape
'o'	Circle
'*'	Star
'^'	Triangle
's'	Square
'D'	Diamond
'x'	Cross

```
plt.scatter(x, y, marker='o', s=100)
```

The `s=` parameter controls the marker size. Larger values produce larger markers.



- Scatter Plots with Regression Lines: One of the most common uses of scatter plots is to determine whether data follows a trend. For example: **More study time** → **Higher exam scores**.

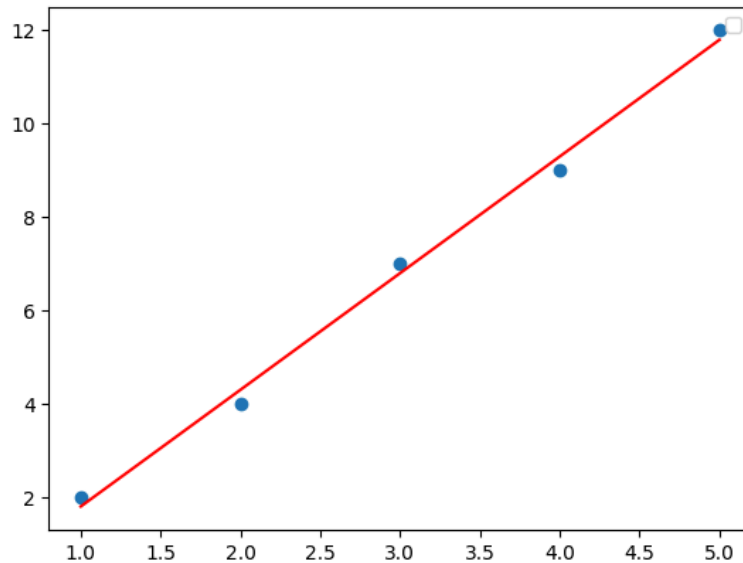
A **Regression Line** attempts to find the best-fitting line through the points.

```
# Regression Line
import numpy as np
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [2, 4, 7, 9, 12]

m, b = np.polyfit(x, y, 1)

plt.plot(x, y, 'o')
# The line is red
plt.plot(x, m*np.array(x) + b, 'r')
plt.legend()
plt.show()
```



In Data Analytics and Machine Learning, regression lines help us identify trends, measure correlation, make predictions, and build predictive models. We will revisit regression in much greater detail when studying Machine Learning and Data Analytics.

2.5. Pie Charts

A Pie Chart shows how a whole is divided into parts. Think of it like slicing a pizza. Each slice represents a percentage of the total.

Pie charts are useful when showing proportions, displaying percentages, or comparing parts of a whole. Examples include: Company budgets; Market share; Population distribution; Time allocation

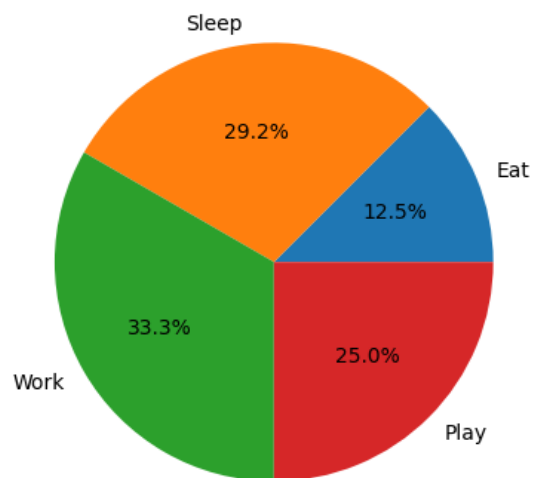
- Normal Pie Chart

```
import matplotlib.pyplot as plt

activities = ["Eat", "Sleep", "Work", "Play"]
slices = [3, 7, 8, 6]

plt.pie(slices, labels=activities,
        autopct="%1.1f%%")
plt.show()
```

The `autopct` parameter automatically displays percentages.



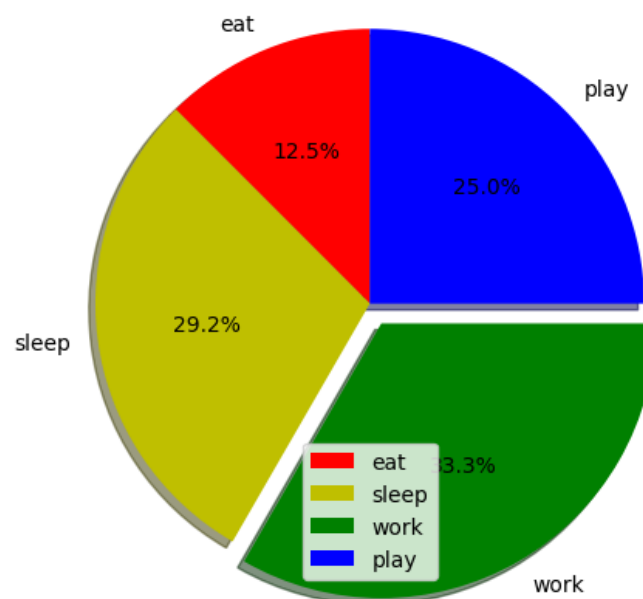
- **Exploded Pie Chart:** Sometimes we want to emphasize a particular category. We can “pull out” one slice from the pie.

```
import matplotlib.pyplot as plt

activities = ['eat', 'sleep', 'work', 'play']
slices = [3, 7, 8, 6]
colors = ['r', 'y', 'g', 'b']

plt.pie(slices, labels=activities, colors=colors,
        startangle=90, shadow=True, explode=(0, 0, 0.1, 0),
        radius=1.2, autopct='%1.1f%%')
plt.legend()
plt.show()
```

Here: `explode=(0,0,0.1,0)` causes the third slice ("work") to move slightly outward. This is known as an **Exploded Pie Chart**.



- Donut Charts: A Donut Chart is simply a Pie Chart with a hole in the middle. It provides the same information while often looking cleaner and more modern.

```
# Donut Chart
import matplotlib.pyplot as plt
```



```

activities = ['eat', 'sleep', 'work', 'play']
slices = [3, 7, 8, 6]
colors = ['r', 'y', 'g', 'b']

# Initialize figure for clean scaling
fig, ax = plt.subplots(figsize=(6, 6))

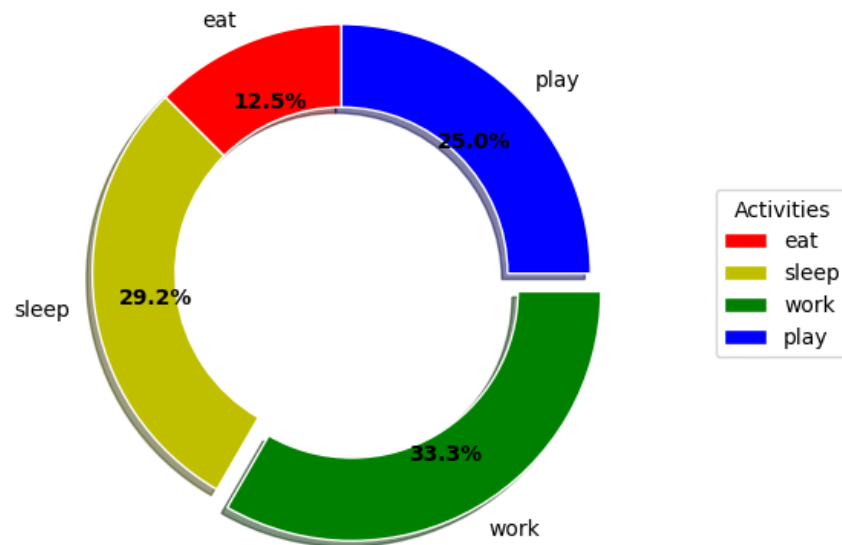
# Create the donut chart
wedges, texts, autotexts = ax.pie(slices,
                                   labels=activities,
                                   colors=colors,
                                   startangle=90,
                                   shadow=True,
                                   explode=(0, 0, 0.1, 0),
                                   radius=1.2,
                                   autopct='%1.1f%%',
                                   # Positions percentages i
                                   # inside the ring
                                   pctdistance=0.75,
                                   # width=0.4 creates the c
                                   # enter donut hole
                                   wedgeprops=dict(width=0.
4, edgecolor='w'))

# Bold the percentage text inside the ring
plt.setp(autotexts, size=10, weight="bold")

# Move the legend outside so it doesn't overlap the chart
ax.legend(wedges, activities, title="Activities",
          loc="center left", bbox_to_anchor=(1.2, 0.5))

plt.tight_layout()
plt.show()

```



Donut charts are very common in: Dashboards; Business reports; Mobile applications

2.6. Histograms

Suppose we have a list of exam scores: 75, 80, 81, 82, 90, 91, 92... . Instead of plotting every score individually, we might want to know: Where most scores occur? Which values are common? Which values are rare?

This is exactly what a histogram does. A histogram groups values into intervals called **bins** and counts how many values fall inside each interval. Think of it as an automated frequency table.

- Basic Histogram

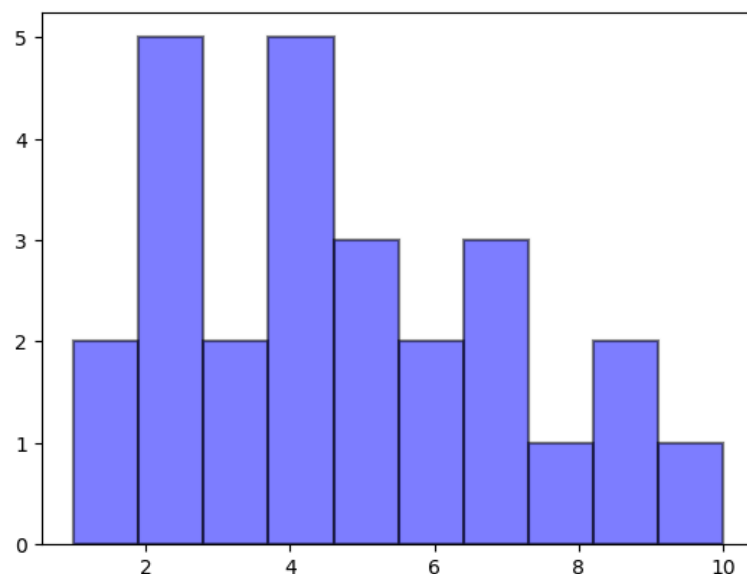
```
import matplotlib.pyplot as plt

x = [2, 1, 6, 4, 2, 4, 8, 9, 4, 2, 4, 10,
     6, 4, 5, 7, 7, 3, 2, 7, 5, 3, 5, 9, 2, 1]

plt.hist(x, bins=10, color='blue', alpha=0.5,
        edgecolor='black', linewidth=1.5)
plt.show()
```

Here: **bins = 10** divides the data into 10 intervals. The height of each bar represents the frequency of values inside that interval.

The parameters **edgecolor='black', linewidth=1.5** are used



Histograms help us understand the **distribution** of data. For example: Is the data centered? Is it skewed? Does it contain multiple peaks? Are there unusual outliers? This becomes extremely important in statistics, Machine Learning, and data analysis.

2.7. Plotting Curves from Equations

Matplotlib isn't limited to plotting existing data. We can also visualize mathematical equations. This allows us to see how functions behave rather than simply reading formulas. For many students, this is where mathematics suddenly becomes much easier to understand.

```

import matplotlib.pyplot as plt
import numpy as np

def plot_equation(equation, x_label, y_label, title, x_range=-10, y_range=10):
    x = np.linspace(x_range, y_range, 100)
    y = eval(equation)

    plt.figure(figsize=(8, 5))
    plt.plot(x, y, label=equation, color="blue", linewidth=2)

    plt.title(title, fontsize=14)
    plt.xlabel(x_label)
    plt.ylabel(y_label)

    plt.axhline(0, color='black', linewidth=0.5, linestyle='--') # X-axis baseline
    plt.axvline(0, color='black', linewidth=0.5, linestyle='--') # Y-axis baseline
    plt.grid(True, linestyle=':', alpha=0.6)
    plt.legend()

    plt.show()

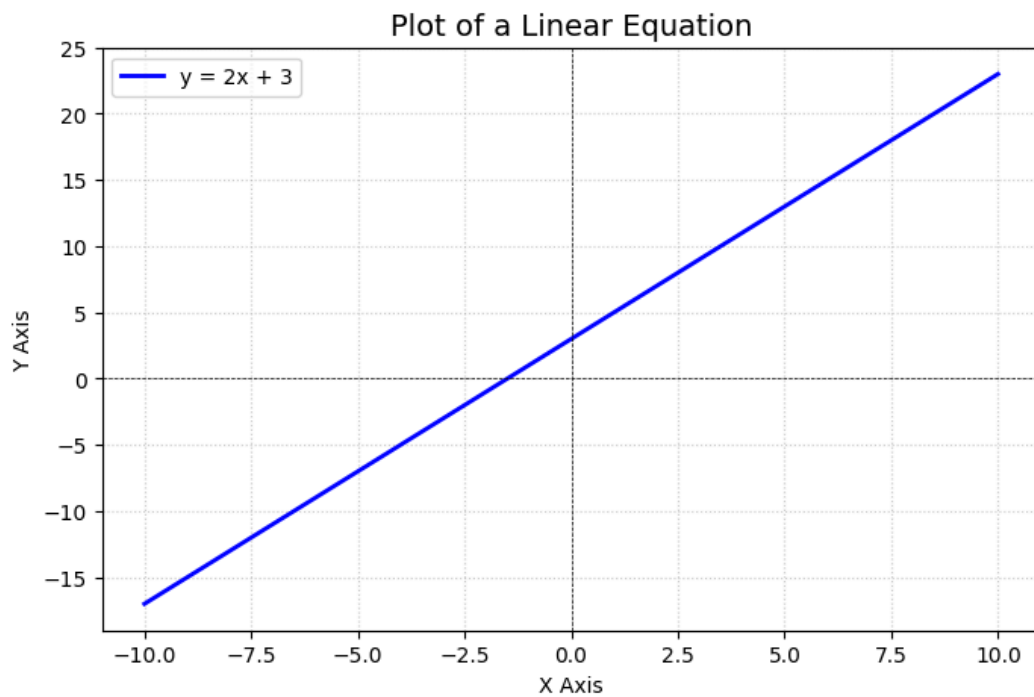
```

- Linear Equation: A linear equation produces a straight line. For example: $y = 2x + 1$. The graph rises at a constant rate.

```

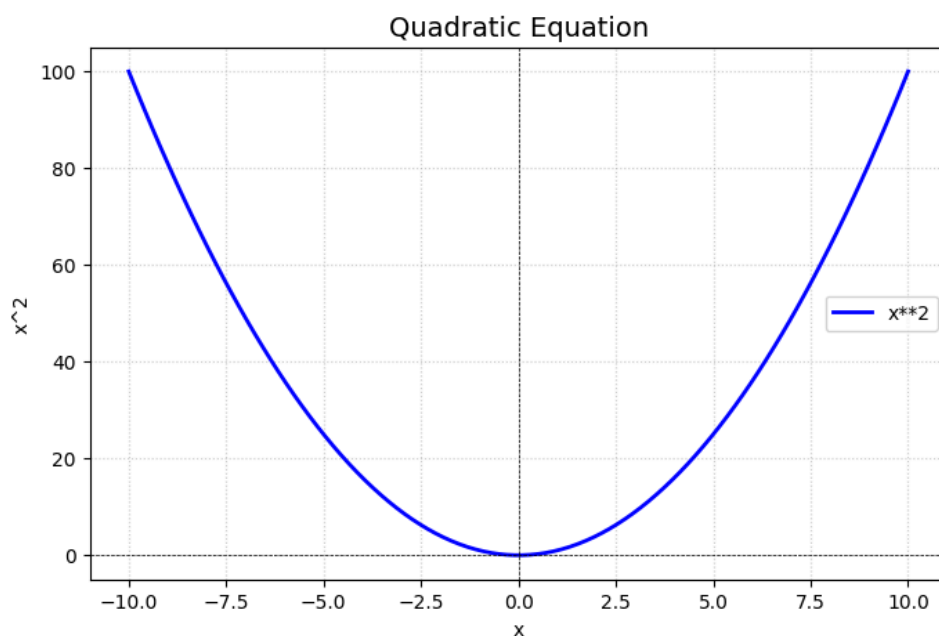
# Plot linear equation
plot_equation("2*x + 3", "x", "2x + 3", "Linear Equation")

```



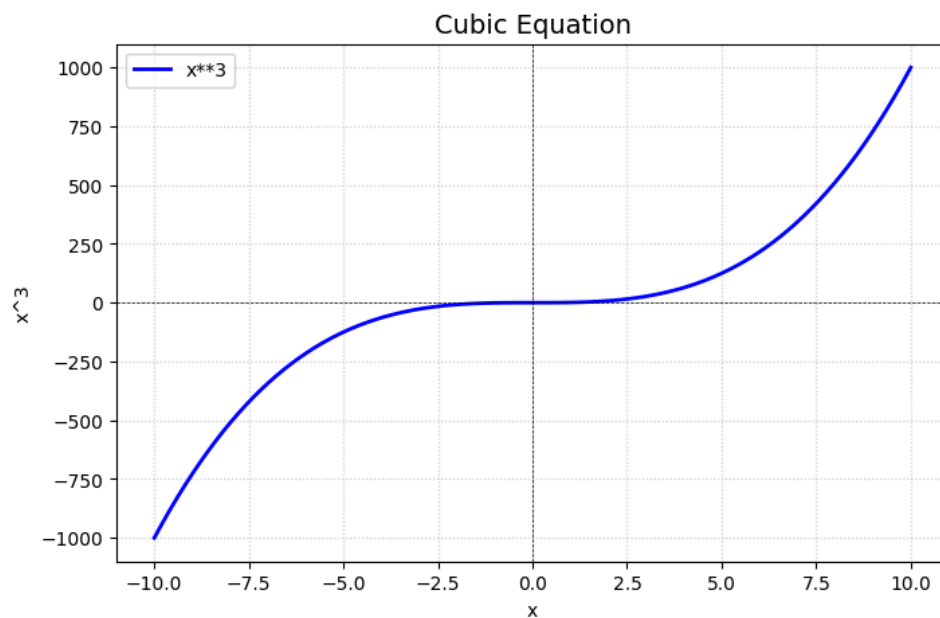
- Quadratic functions create a parabola. Example: $y = x^2$. This famous U-shaped curve appears everywhere in mathematics, physics, and optimization problems.

```
# Plot quadratic equation  
plot_equation("x**2", "x", "x^2", "Quadratic Equation")
```



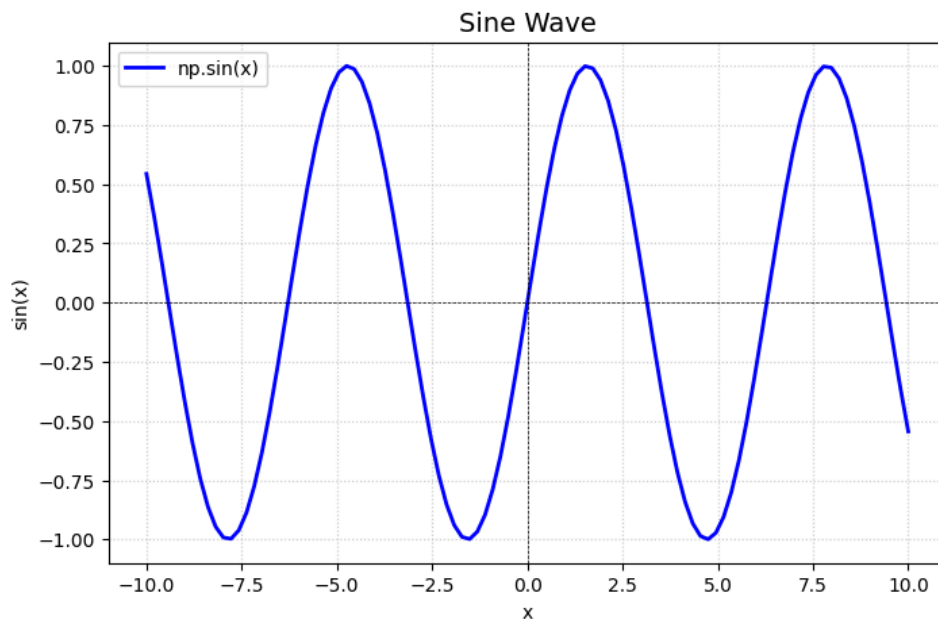
- Cubic functions often produce an S-shaped curve. Example: $y = x^3$. Unlike quadratics, cubic functions can contain multiple turning points.

```
# Plot cubic equation
plot_equation("x**3", "x", "x^3", "Cubic Equation")
```



- Trigonometric Functions: One of the most important mathematical curves is the sine wave. Example: $y = \sin(x)$. This waveform appears throughout science and engineering like sound waves, light waves, alternating current (AC), signal processing, or communication systems. Because of its repeating nature, it is known as a **periodic function**.

```
# Plot sine wave
plot_equation("np.sin(x)", "x", "sin(x)", "Sine Wave")
```



A mathematical equation tells us what a function does. A graph shows us how it behaves. By plotting equations, we can visualize mathematical concepts, identify maximum and minimum points, or build intuition for calculus and Machine Learning.

And that is one of Matplotlib's greatest strengths: It allows us to transform abstract mathematics into something we can immediately see and understand.

3. Drawing Multiple Plots

So far, every graph we created occupied the entire figure. But in real-world data analysis, that is rarely enough. Imagine trying to compare: Sales vs Profit; Temperature vs Humidity; Accuracy vs Loss; Multiple Machine Learning models.

Displaying these graphs one at a time quickly becomes inconvenient. Instead, Matplotlib allows us to place multiple plots inside the same figure using **Subplots**. This is one of the most important visualization techniques because it allows us to compare multiple datasets side-by-side without opening dozens of separate windows.

There are several ways to create subplots:

```
plt.subplot(...)
```

or:

```
fig, ax = plt.subplots(...)
```

The second approach is generally preferred for larger and more professional projects because it provides greater control over each graph.

3.1. Drawing Subplots

A subplot is simply a graph placed inside a larger figure. Think of it as dividing a canvas into smaller sections.

- **Two Subplots:** The following example creates two plots placed side-by-side.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

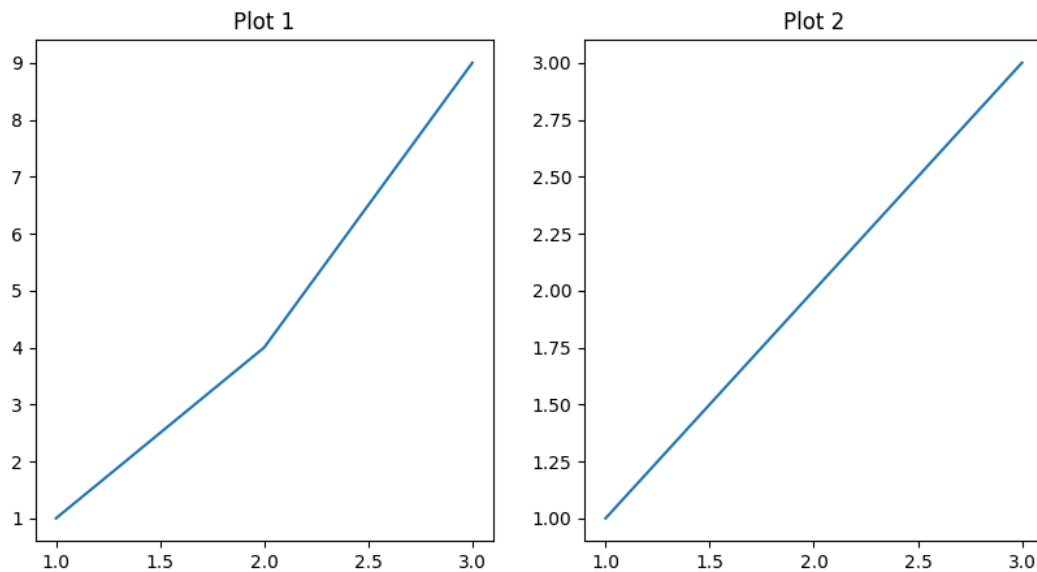
plt.subplot(1, 2, 1)
plt.plot([1, 2, 3], [1, 4, 9])
plt.title("Plot 1")

plt.subplot(1, 2, 2)
plt.plot([1, 2, 3], [1, 2, 3])
plt.title("Plot 2")

plt.show()
```

The syntax is `plt.subplot(rows, columns, position)`. So:

- `plt.subplot(1, 2, 1)` means 1 row; 2 columns; First position
- `plt.subplot(1, 2, 2)` creates the second plot.



- Four Subplots: We can also create a grid layout.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.subplot(2, 2, 1)
plt.plot([1, 2, 3])

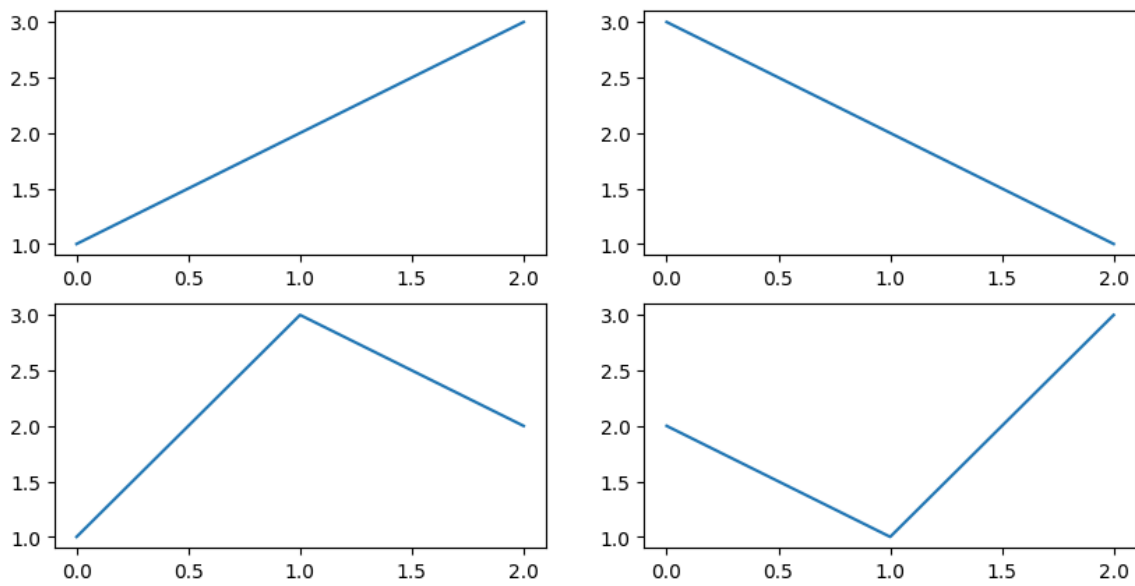
plt.subplot(2, 2, 2)
plt.plot([3, 2, 1])

plt.subplot(2, 2, 3)
plt.plot([1, 3, 2])

plt.subplot(2, 2, 4)
plt.plot([2, 1, 3])

plt.show()
```

A layout like this is commonly used for comparing multiple experiments, showing different visualizations of the same dataset, machine learning model evaluation, or dashboard-style reports



The Modern Approach: fig and axes

As projects become larger, the `fig, ax` approach becomes easier to manage.

```

import matplotlib.pyplot as plt

# Create a 2x2 grid of subplots
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

# Data for the four plots
plot_data = [([1, 2, 3], [1, 4, 9], "Plot 1"),
              ([1, 2, 3], [1, 2, 3], "Plot 2"),
              ([1, 2, 3], [3, 1, 2], "Plot 3"),
              ([1, 2, 3], [2, 3, 1], "Plot 4")]

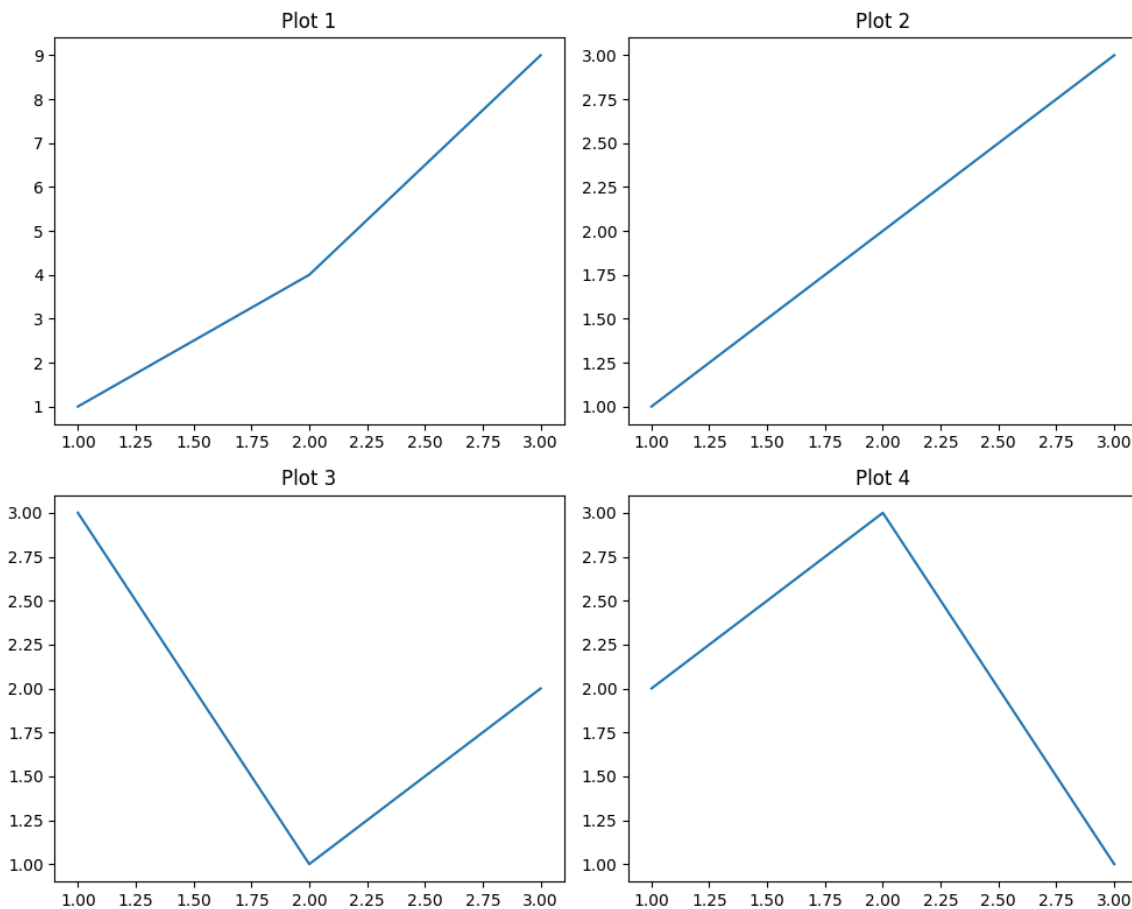
# Loop through the axes and plot the data
plot_index = 0
for r in range(2):
    for c in range(2):
        x_vals, y_vals, title = plot_data[plot_index]
        axes[r, c].plot(x_vals, y_vals)
        axes[r, c].set_title(title)
        plot_index += 1

# Adjust layout to prevent overlapping titles/labels
plt.tight_layout()
plt.show()

```

Using a loop to generate subplots is the best way to eliminate repetitive code, especially when you need to visualize multiple variables or data categories side by side.

Think of **fig** as the entire canvas. And **axes** as individual graph areas inside that canvas. This style is heavily used in professional Data Science projects.



3.2. Drawing Different Types of Plots Together

One of the biggest advantages of subplots is that each subplot can contain a completely different graph. For example:

```
import matplotlib.pyplot as plt

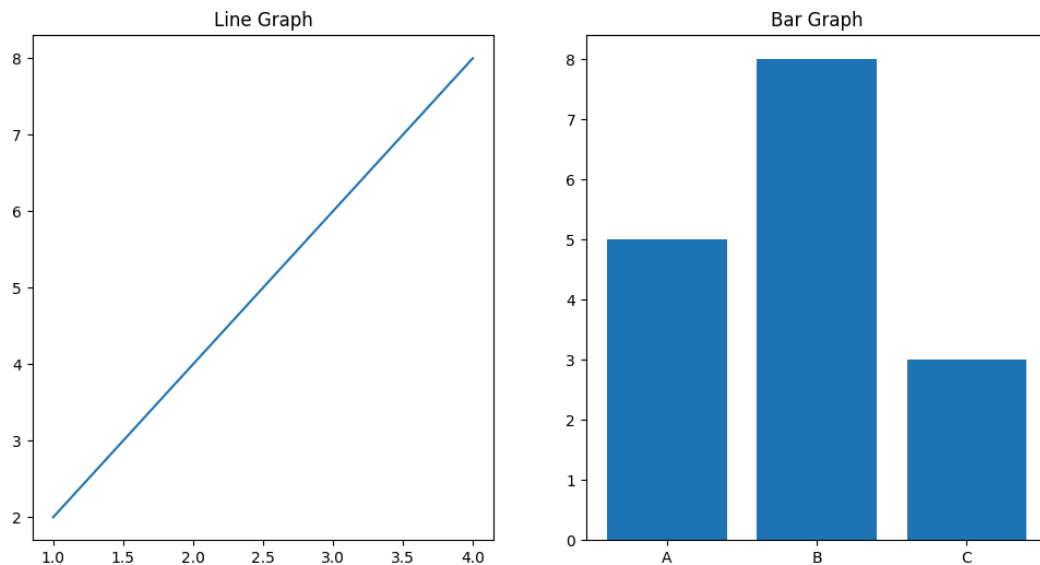
plt.figure(figsize=(10, 8))
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

axes[0].plot([1, 2, 3, 4], [2, 4, 6, 8])
axes[0].set_title("Line Graph")

axes[1].bar(
    ["A", "B", "C"],
    [5, 8, 3])
```

```
)
axes[1].set_title("Bar Graph")

plt.show()
```



The left graph is a line chart. The right graph is a bar chart.

There is no requirement that all subplots use the same visualization type. In fact, combining different graph styles often provides a much better understanding of the data. For example:

Plot Type	Purpose
Line Plot	Trends
Scatter Plot	Relationships
Histogram	Distribution
Bar Chart	Category Comparison
Pie Chart	Proportions

Mixing them together creates richer reports and dashboards.



A Quick Note: Multiple Y-Axes with `twinx()`

Sometimes two datasets share the same x-axis but use completely different scales. For example: Rainfall (0–150 mm); Temperature (0–40 °C)

If we plot them on the same y-axis, one dataset may dominate the graph while the other becomes difficult to read.

Matplotlib provides the `twinx()` method, which creates a **second y-axis** on the right side of the figure while sharing the same x-axis. This is commonly used for:

- Climate graphs (Rainfall + Temperature)
- Stock Price + Trading Volume
- Population + GDP
- Sales + Profit

A classic example is a climate chart where rainfall is shown as bars and temperature is shown as a line on top of those bars. Each dataset uses its own y-axis scale while sharing the same months along the x-axis.

```
# Advanced Example: Create a classic climate graph
# (like a temperature and
# rainfall plot)
# with bars at the bottom and a line above them,

import matplotlib.pyplot as plt

# 1. Define your climate data (e.g., 6 months of data)
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
rainfall_mm = [75, 88, 110, 65, 45, 20]      # Bar data (Lower values)
temp_celsius = [12, 14, 18, 22, 26, 30]      # Line data (Higher values)

# 2. Initialize the single plot and the primary Y-axis (Left)
fig, ax1 = plt.subplots(figsize=(9, 6))
```

```

# 3. Draw the Rainfall Bars on the primary axis
# Set alpha (transparency) so grid lines or data do
n't completely hide
bars = ax1.bar(months, rainfall_mm, color="royalblue",
alpha=0.6, width=0.5, label="Rainfall (mm)")
ax1.set_xlabel("Months", fontsize=12, fontweight="bold")
ax1.set_ylabel("Rainfall (mm)", color="royalblue",
fontsize=12, fontweight="bold")
ax1.tick_params(axis="y", labelcolor="royalblue")
ax1.set_ylim(0, 150) # Set limit so bars stay neatly
at the bottom half

# 4. Create the secondary Y-axis (Right) sharing the
same X-axis
ax2 = ax1.twinx()

# 5. Draw the Temperature Line on the secondary axis
line = ax2.plot(months, temp_celsius, color="crimson",
marker="o", linewidth=2.5, label="Temperature
(°C)")
ax2.set_ylabel("Temperature (°C)", color="crimson",
fontsize=12, fontweight="bold")
ax2.tick_params(axis="y", labelcolor="crimson")
ax2.set_ylim(0, 40) # Adjust scale so the line hovers
elegantly above the bars

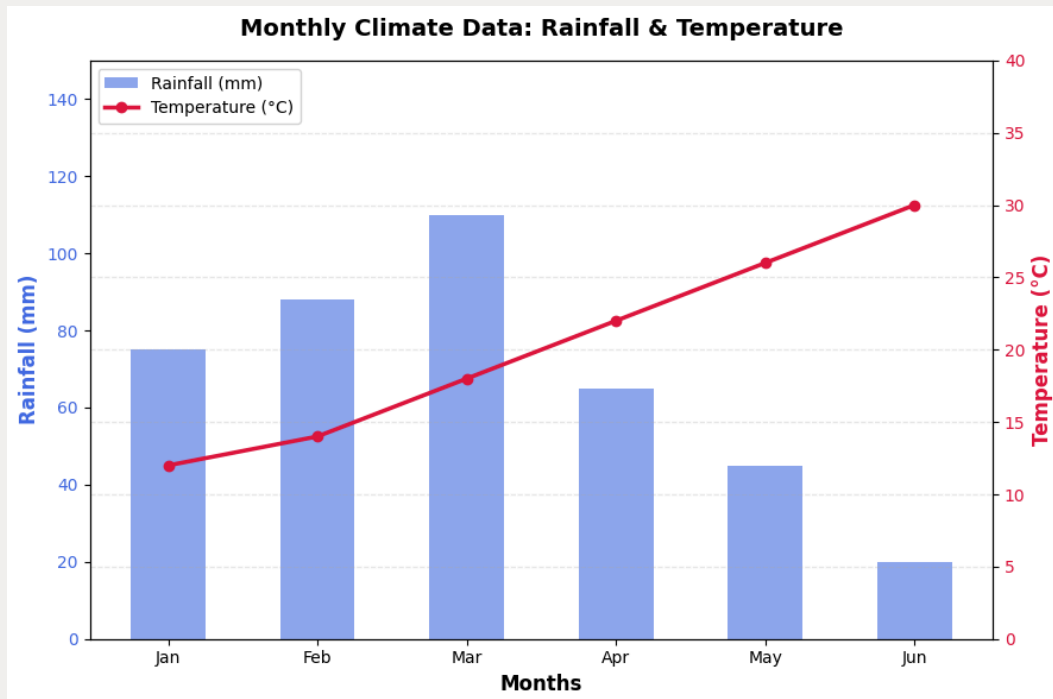
# 6. Add a clean, combined legend for both plot types
# Since they are on different axes, we manually combine
their labels
lines_labels = [bars, line[0]]
labels = [l.get_label() for l in lines_labels]
ax1.legend(lines_labels, labels, loc="upper left")

# 7. Add Title and finalize layout
plt.title("Monthly Climate Data: Rainfall & Temperature")

```

```
ture", fontsize=14, fontweight="bold", pad=15)
plt.grid(axis='y', linestyle='--', alpha=0.3) # Sub
tle horizontal gridlines
plt.tight_layout()

# 8. Render the single figure
plt.show()
```



Quick Note: While `twinx()` is extremely useful, avoid overusing it. Too many axes on the same figure can make graphs difficult to interpret.

3.3. Titles

As figures become larger, proper labeling becomes increasingly important. Matplotlib supports both **subplot titles** and **figure titles**

- Subplot Titles: Each graph can have its own title.

```
axes[0].set_title("Sales")
axes[1].set_title("Profit")
```


These titles describe individual plots.

- Figure Title (Super Title): Sometimes we want a title for the entire figure. This is called a **Super Title**.

```
fig.suptitle("Company Performance Report")
```

- Example:

```
# Plot Sales vs Profit
import matplotlib.pyplot as plt

# Data
sales = [100, 150, 120, 200, 180, 130]
profit = [50, 70, 60, 90, 80, 40]

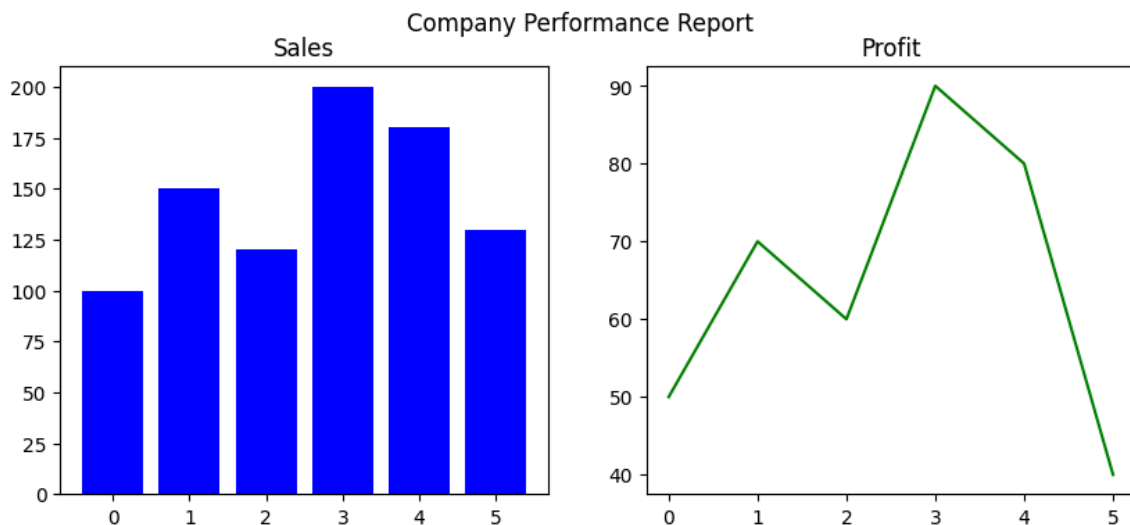
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

# Sales Plot
axes[0].bar(range(len(sales)), sales, color="blue")
axes[0].set_title("Sales")

axes[1].plot(range(len(profit)), profit, color="green")
axes[1].set_title("Profit")

fig.suptitle("Company Performance Report")
```

The super title provides context for the entire figure, while subplot titles describe each individual graph.



4. Other Plots

So far, we have covered the most commonly used visualizations in Matplotlib. However, Matplotlib supports many other plot types that are frequently used in statistics, data analytics, and scientific computing.

The following are some of the most popular advanced visualizations.

4.1. Box Plot

A **Box Plot** (sometimes called a Box-and-Whisker Plot) is used to visualize the distribution of data.

Unlike a histogram, which groups values into bins, a box plot summarizes data using: Minimum value; First Quartile (Q1); Median; Third Quartile (Q3); Maximum value

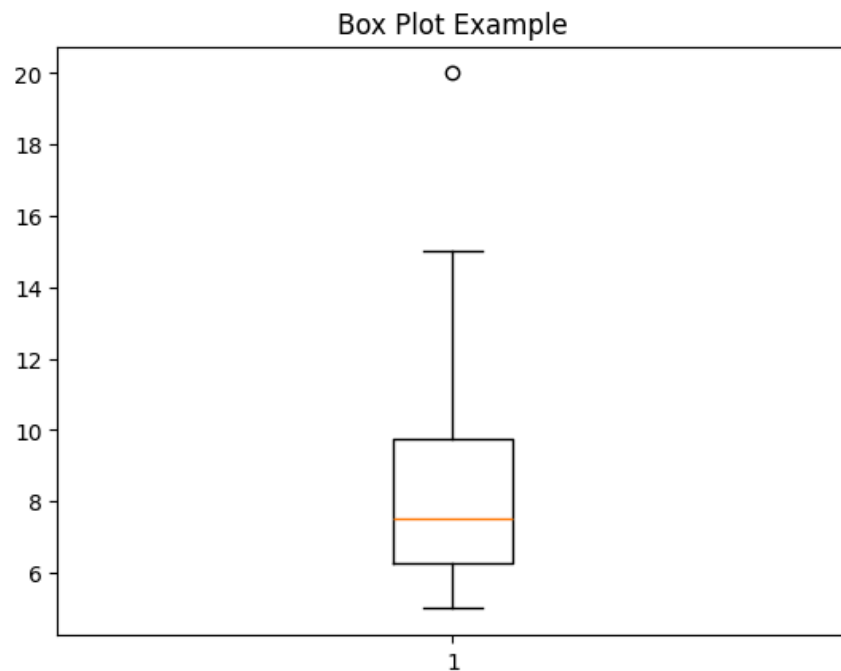
Box plots are commonly used in statistics and exploratory data analysis. It is especially useful for identifying data spread, skewness, outliers

```
import matplotlib.pyplot as plt

data = [7, 8, 5, 6, 9, 10, 15, 20, 6, 7]

plt.boxplot(data)
```

```
plt.title("Box Plot Example")
plt.show()
```



4.2. Heatmaps

A **Heatmap** visualizes data using colors. Instead of bars or points, values are represented by different color intensities. For example:

- Low Value → Light Color
- High Value → Dark Color

Heatmaps are commonly used for correlation matrices, confusion matrices, scientific measurements, and geographic and climate data

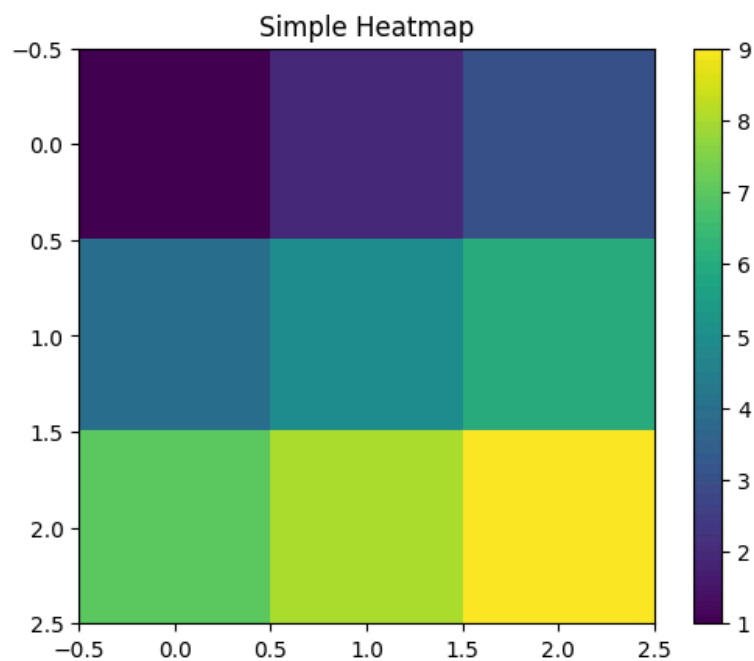
```
import matplotlib.pyplot as plt

data = [[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]]

plt.imshow(data)
```

```
plt.colorbar()

plt.title("Simple Heatmap")
plt.show()
```



We will later see that Seaborn provides much more powerful heatmap support.

4.3. 3D Plots

Most graphs we have seen so far are two-dimensional: (x-axis, y-axis). But some datasets require an additional dimension.

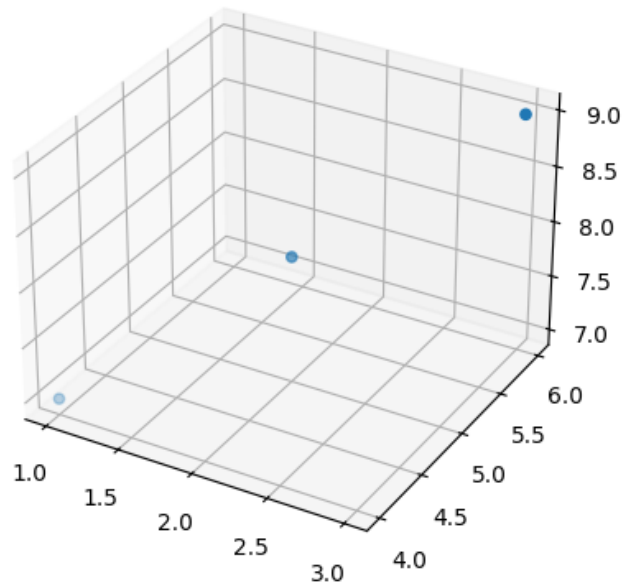
Matplotlib allows us to create **3D plots** using an additional z-axis → (x-axis, y-axis, z-axis)

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure()

ax = fig.add_subplot(111, projection='3d')
```

```
ax.scatter([1, 2, 3],  
          [4, 5, 6],  
          [7, 8, 9])  
plt.show()
```



3D visualizations are often used in engineering, physics, scientific simulations, machine Learning visualizations, and geographic modeling.

Matplotlib contains dozens of additional plot types beyond those covered in this chapter. In practice, however, line plots, scatter plots, bar charts, histograms, box plots, and heatmaps account for the vast majority of visualizations used in data science and analytics. Understanding these core plots will already allow you to visualize most real-world datasets effectively.

Summary

Matplotlib is much more than a tool for drawing a single graph. In this section, we learned how to:

- Create line graphs
- Customize graph appearance

- Create scatter plots
- Build pie charts and histograms
- Plot mathematical equations
- Display multiple plots in the same figure
- Use subplot grids
- Combine different visualization types
- Add subplot titles and figure-wide titles

These skills form the foundation of data visualization in Python.

However, Matplotlib can sometimes require a lot of code for graphs that should be simple. This is where **Seaborn** enters the picture. Built on top of Matplotlib, Seaborn provides a cleaner interface, better default styles, and powerful statistical visualization tools with significantly less code.

In the next section, we will explore how Seaborn makes beautiful visualizations even easier to create.

References

<https://matplotlib.org/>

<https://www.geeksforgeeks.org/python/matplotlib-tutorial/>

Next Section:

[!\[\]\(e1d6102fe77919492c04879c8450f1f5_img.jpg\) 32.1. Python: Seaborn](#)